KARADENİZ TEKNİK ÜNİVERSİTESİ — BİLGİSAYAR BİLİMLERİ

Hafta 7 Laboratuvar Föyü

React III: API Tüketimi ve Yönlendirme — Etkinlik/Bilet Sisteminin Ön Yüzünü Gerçek Sunucuya Bağlayın

Doç. Dr. Tolga Berber · https://avesis.ktu.edu.tr/tberber

Tahmini süre: 2 saat
Gereksinimler: Node.js 24 LTS ve Python 3.12 (asgari) kurulu bir bilgisayar, bir metin editörü (VS Code önerilir), güncel bir tarayıcı (Chrome/Firefox/Edge, DevTools erişimiyle), Hafta 4–5'ten tamamladığınız `istemci-react` klasörü, Hafta 6'dan tamamladığınız `sunucu` klasörü — `react-router` paketini indirmek için internet bağlantısı gerekir

Öğrenme fazları: 1 • Ön-öğretim | 2 • İçerik | 3 • Katılım | 4 • Değerlendirme | 5 • İzleme

1 • Ön-öğretim — Neden Önemli?

Geçen hafta iki ayrı proje bıraktınız: `istemci-react` tarayıcıda güzel görünüyordu ama hâlâ Hafta 3'ün üç sabit etkinliğini gösteriyordu; `sunucu` ise `curl` ile mükemmel çalışıyordu ama hiçbir tarayıcı sayfası onu kullanmıyordu. Bugün iki saat sonra bu iki proje aynı anda iki terminalde açık olacak: React uygulamanız `fetch` ile gerçek sunucudan veri çekecek, önce tarayıcı konsolunda kızıl bir CORS hatasıyla karşılaşacak, sonra tek bir kod bloğuyla bu engeli aşacak ve uygulamanız artık tek bir ekran değil, adres çubuğunda gerçekten değişen dört ayrı sayfaya (Etkinlikler, Etkinlik Detayı, Biletlerim, 404) bölünecek — bir koltuğu iki ayrı sekmeden aynı anda almaya çalıştığınızda sunucudan gelen gerçek `409` hatasını ekranda göreceksiniz.

1 • Ön-öğretim — Ön Bilgi

Bu föy Hafta 7 ders notunun dört ünitesine dayanır: Ünite 1 (`fetch` ve Tiplenmiş İstemci), Ünite 2 (Yükleme/Hata/Boş Durumları), Ünite 3 (CORS İlk Temas) ve Ünite 4 (React Router: Sayfalar, Parametreli Rotalar). Hafta 3 Ünite 4'ün `Promise`/`async`/`await` bilgisini (`fetch` de birden yeni bir Promise kaynağıdır); Hafta 5 Ünite 1'in `useEffect`+bağımlılık dizisi+temizleme fonksiyonu bilgisini ve Ünite 4'ün özel hook kalıbını (`useSepet`, `useGeriSayim`); Hafta 1 Ünite 3'ün SPA/SSR ayrımını ve Ünite 4'ün DevTools deneyimini hazır bilgi olarak kullanacaksınız. Hafta 6'nın tüm rotaları, durum kodları ve camelCase kararı bu hafta değiştirilmeden, olduğu gibi tüketilir — sunucu tarafında bugünkü tek değişiklik `app/main.py`'ye eklenecek bir CORS bloğudur.

Föyü şöyle kullanın: her adımda önce *Ne yapacağız?* kutusundaki görevi kendi projenize **elle yazarak** tamamlayın, sonra terminaldeki/tarayıcıdaki çıktınızı *Beklenen sonuç* kutusuyla karşılaştırın. Adımlardaki kod kutuları `kod_ornekleri/hafta07/istemci/` ve `kod_ornekleri/hafta07/sunucu/` klasörlerindeki referans dosyalardan gelir — bunlar hedef/"doğru cevap" niteliğindedir; siz bu kodu kopyala-yapıştır yapmak yerine kendi dosyalarınıza yazarsınız, sonucu terminalde/tarayıcıda karşılaştırırsınız. Birkaç adımda bir gelen *Kontrol Noktası*, o ana kadar yazdıklarınızı bir arada toparlamanıza yarar.

Bu hafta iki proje birlikte çalışır: bugünün tamamı boyunca iki terminal açık kalacak — Terminal A `sunucu` klasöründe (port `8000`), Terminal B `istemci-react` klasöründe (port `5173`). `sunucu`

klasörünüz Hafta 6'dan değişmeden devam eder (bugün yalnızca `app/main.py`'ye bir CORS bloğu ekleyeceksiniz); `istemci-react` klasörünüz Hafta 4–5'ten aynen devam eder. Yeni bir proje oluşturmayacaksınız.

1 `fetch` ve Tiplenmiş İstemci

(bkz. Ders notu Hafta 7, Ünite 1 — `fetch` ve Tiplenmiş İstemci.)

Adım 1: İki terminali açıp iki projenizin hâlâ çalıştığını doğrulayın

Ne yapacağız? Terminal A'da `sunucu` klasörüne girip sanal ortamı etkinleştirip `uvicorn`'u başlatacak; Terminal B'de `istemci-react` klasöründe `npm run dev`'i başlatacak ve tarayıcıda Hafta 5'te bıraktığınız hâliyle (3 sabit etkinlik kartı, koltuk ızgarası, sepet) hâlâ çalıştığını doğrulayacaksınız.

Neden? Bugünkü tüm adımlar bu iki projenin üzerine inşa edilecek — hiçbiri yeniden oluşturulmayacak. İki projeyi de çalışır durumda görmeden devam etmek, bugün karşılaşacağınız bir hatanın yeni kodunuzdan mı yoksa eksik bir kurulumdan mı geldiğini ayırt etmenizi zorlaştırır.

Terminal

```
$ cd sunucu
$ source .venv/bin/activate
$ uvicorn app.main:app --reload
```

Terminal

```
$ cd istemci-react
$ npm run dev
```

Beklenen sonuç: Terminal A'da "Uvicorn running on http://127.0.0.1:8000" satırı, Terminal B'de Vite'ın başlangıç bildirimi görünür; tarayıcıda `http://localhost:5173` adresinde Hafta 5'teki hâliyle uygulamanız (3 etkinlik kartı, koltuk planı, sepet) açılır — bugün henüz hiçbir sunucu bağlantısı yoktur.

Beklenen Çıktı

```
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Application startup complete.
```

Beklenen Çıktı

```
  VITE v8.2.2  ready in 108 ms

  ➜  Local:   http://localhost:5173/
```

İpucu

Bugünün geri kalanında her iki terminali de açık bırakın — Terminal A `Ctrl+C` ile durdurulmadıkça çalışmaya devam eder, Terminal B'nin Vite HMR'ı her kaydettiğinizde sayfayı otomatik yeniler.

Adım 2: `src/api/istemci.ts` dosyasını yazın

Ne yapacağız? `istemci-react` projenizde `src/api` klasörünü oluşturup içine `istemci.ts` dosyasını ekleyecek; `TABAN_URL` sabitini, `ApiHatasi` sınıfını ve `apiGet/apiPost/apiPatch/apiDelete` fonksiyonlarını yazacaksınız.

Neden? Her bileşende `fetch` + `response.ok` kontrolü + `response.json()` üçlüsünü tekrar tekrar yazmak yerine, bu üç adımı tek bir yerde toplayan tiplenmiş bir istemci katmanı kuruyorsunuz. `response.ok` `false` olduğunda (`404/409` gibi) bu fonksiyonlar `ApiHatasi` sınıfından bir örnek fırlatır — bu, bir ağ hatasıyla (sunucuya hiç ulaşılamaması) bir HTTP hata cevabı (sunucunun bir cevap üretmiş olması) arasındaki kritik ayrımı tek bir yerde toplar. `apiGet<T>`'nin jenerik tip parametresi `T`, Hafta 3'ün `Liste<T>` kalıbının bu haftaki karşılığıdır: fonksiyonun ne döneceğini kullanım anında (`apiGet<Etkinlik[]>(...)` gibi) siz belirlersiniz.

```ts
// Hf.6 sunucusunun adresi tek bir sabitte tutulur; HER fetch çağrısı bu
// sabiti kullanır. (İpucu: gerçek projelerde bu adres bir .env dosyasından
// okunur — ortam değişkeni ile yapılandırma Hf.12'nin konusudur.)
export const TABAN_URL = 'http://127.0.0.1:8000';
```

```ts
// response.ok false olduğunda apiGet/apiPost bu sınıftan bir örnek fırlatır;
// çağıran kod `e instanceof ApiHatasi && e.status === 409` gibi durum
// koduna göre ayrım yapabilir.
export class ApiHatasi extends Error {
  status: number;
  detail: unknown;

  constructor(status: number, detail: unknown) {
    super(`API hatası: ${status}`);
    this.name = 'ApiHatasi';
    this.status = status;
    this.detail = detail;
  }
}
```

```ts
export async function apiGet<T>(
  yol: string,
  { signal }: IstekSecenekleri = {},
): Promise<T> {
  const response = await fetch(`${TABAN_URL}${yol}`, { signal });
  return cevabiIsle<T>(response);
}
```

```ts
export async function apiPost<TIn, TOut>(
  yol: string,
  govde: TIn,
  { signal }: IstekSecenekleri = {},
): Promise<TOut> {
  const response = await fetch(`${TABAN_URL}${yol}`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(govde),
    signal,
  });
  return cevabiIsle<TOut>(response);
}
```

Adım 2 (devamı)

```
>_ Terminal
$ npx tsc -b
```

Beklenen sonuç: Komut hatasız, hiçbir çıktı vermeden biter — bu dosyayı henüz hiçbir bileşen kullanmıyor, tarayıcıda görünür bir değişiklik yoktur; `apiPatch`/`apiDelete` fonksiyonlarını da aynı `cevabiIsle` yardımcı fonksiyonu üzerine, dosyanın geri kalanına bakarak kendiniz tamamlayın.

Ortak Hata / Tuzak

`TABAN_URL`'i her `fetch` çağrısında yeniden yazmak ("sabit kodlanmış" URL'ler) yaygın bir hatadır — sunucu adresi değiştiğinde (ör. Hafta 14'ün dağıtımında) onlarca satırı tek tek güncellemeniz gerekir. `TABAN_URL` sabitini tek bir yerde tutup her çağrının bu sabiti kullanması, bu riski ortadan kaldırır.

Adım 3: Varlık modüllerini (`etkinlikler.ts`, `koltuklar.ts`, `biletler.ts`) yazın

Ne yapacağız? `src/api/etkinlikler.ts` dosyasını oluşturup `etkinlikleriGetir`/`etkinlikGetir` fonksiyonlarını; `src/api/koltuklar.ts` dosyasını oluşturup `koltuklariGetir`/`koltukGuncelle` fonksiyonlarını; `src/api/biletler.ts` dosyasını oluşturup `BiletOlusturGirdisi` arayüzünü ve `biletOlustur`/ `biletleriGetir` fonksiyonlarını yazacaksınız.

Neden? Her dosya, Hafta 6'nın ilgili `APIRouter`'ına (`app/rotalar/etkinlikler.py` vb.) birebir karşılık gelir — Hafta 6'nın "her varlık kendi dosyasında" disiplininin ön yüz karşılığıdır. `apiGet<Etkinlik[]>('/etkinlikler')` çağrısının dönüş tipi Hafta 3'ün `interface Etkinlik`'idir — hiçbir `snakeToCamel(...)` dönüşüm fonksiyonu yazmanız gerekmez, çünkü Hafta 6 sunucusu zaten camelCase JSON döner. Bu, iki farklı dilde (TypeScript/Python) tanımlanmış iki modelin, Hafta 6'nın OpenAPI şemasında nasıl buluştuğunun somut kanıtıdır.

```ts
// GET /etkinlikler → Etkinlik[]; mekan verilirse sunucu tarafında filtrelenir.
export function etkinlikleriGetir(
  mekan?: string,
  signal?: AbortSignal,
): Promise<Etkinlik[]> {
  const sorgu = mekan ? `?mekan=${encodeURIComponent(mekan)}` : '';
  return apiGet<Etkinlik[]>(`/etkinlikler${sorgu}`, { signal });
}

// GET /etkinlikler/{id} → Etkinlik; bulunamazsa apiGet bir ApiHatasi (404) fırlatır.
export function etkinlikGetir(id: number, signal?: AbortSignal): Promise<Etkinlik> {
  return apiGet<Etkinlik>(`/etkinlikler/${id}`, { signal });
}
```

```ts
// GET /etkinlikler/{etkinlikId}/koltuklar → Koltuk[]; durum verilirse
// sunucu tarafında filtrelenir (ör. yalnızca boş koltuklar).
export function koltuklariGetir(
  etkinlikId: number,
  durum?: KoltukDurumu,
  signal?: AbortSignal,
): Promise<Koltuk[]> {
```

Adım 3 (devamı)

```
  const sorgu = durum ? `?durum=${durum}` : '';
  return apiGet<Koltuk[]>(`/etkinlikler/${etkinlikId}/koltuklar${sorgu}`, {
    signal,
  });
}

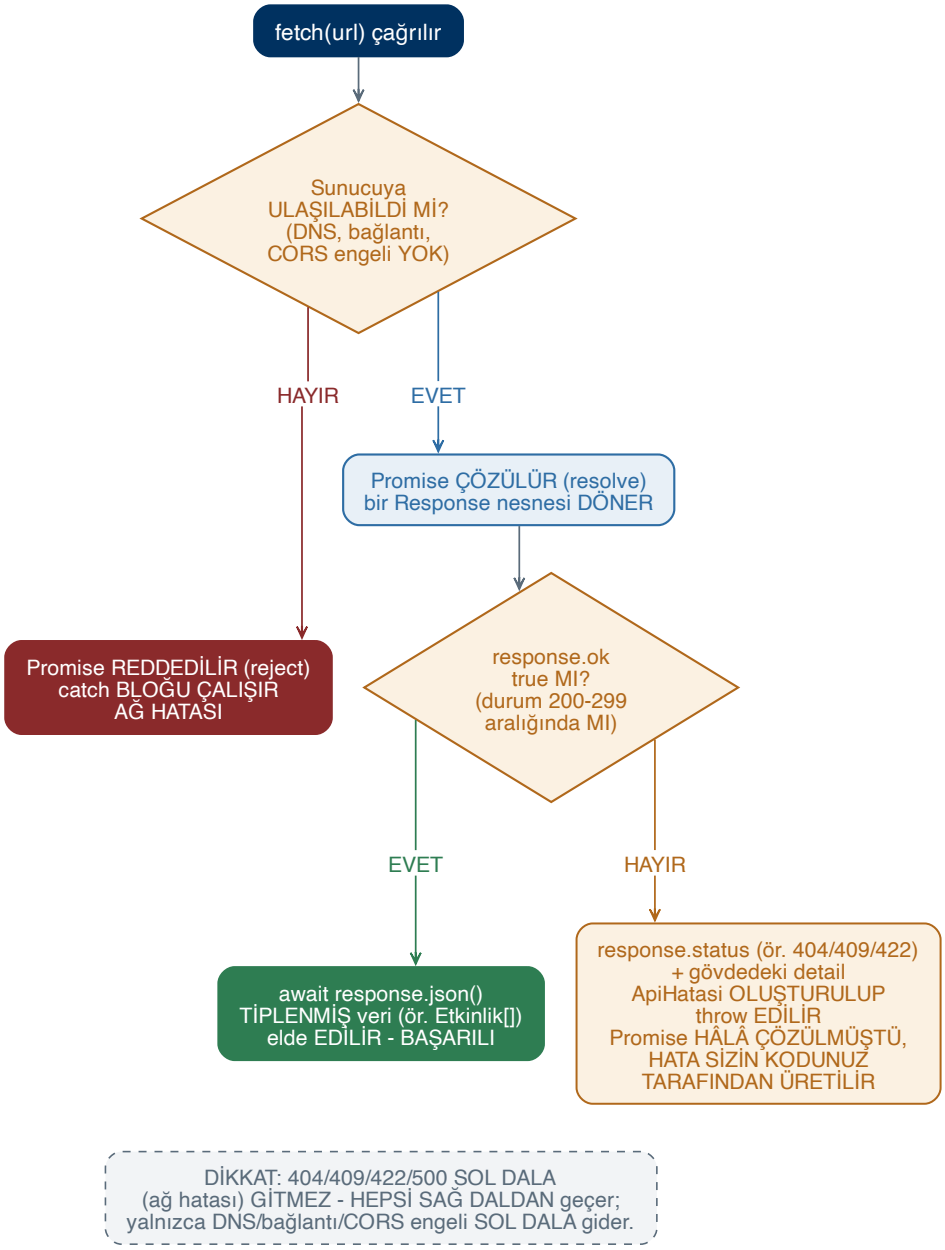
// PATCH /koltuklar/{id} — yalnızca durum alanını günceller; koltuk başka
// biri tarafından zaten alınmışsa sunucu bir 404/409 dönebilir (bu
// dosyanın kendisi bunu YAKALAMAZ, çağıran kod — EtkinlikDetaySayfasi —
// ApiHatasi'nı yakalayıp iyimser değişikliği geri alır).
export function koltukGuncelle(id: number, durum: KoltukDurumu): Promise<Koltuk> {
  return apiPatch<{ durum: KoltukDurumu }, Koltuk>(`/koltuklar/${id}`, { durum });
}
```

```
// POST /biletler gövdesi — sunucunun BiletOlustur modeliyle (koltukId,
// kullaniciAdi) birebir örtüşür.
export interface BiletOlusturGirdisi {
  koltukId: number;
  kullaniciAdi: string;
}

// POST /biletler → Bilet (201); koltuk zaten doluysa sunucu 409 döner,
// apiPost bunu bir ApiHatasi olarak fırlatır (çağıran kod BiletFormu'nda
// yakalanır).
export function biletOlustur(girdi: BiletOlusturGirdisi): Promise<Bilet> {
  return apiPost<BiletOlusturGirdisi, Bilet>('/biletler', girdi);
}

// GET /biletler?kullaniciAdi=... → o kullanıcının biletleri.
export function biletleriGetir(
  kullaniciAdi: string,
  signal?: AbortSignal,
): Promise<Bilet[]> {
  return apiGet<Bilet[]>(
    `/biletler?kullaniciAdi=${encodeURIComponent(kullaniciAdi)}`,
    { signal },
  );
}
```

Adım 3 (devamı)




```
>_ Terminal

$ npx tsc -b
```

Beklenen sonuç: Komut yine hatasız biter. `src/veri.ts` dosyasını henüz silmeyin — `App.tsx` hâlâ ona bağımlı; Bölüm 4'te `App.tsx`'i yeniden yazarken kaldıracaksınız.

Kontrol Noktası — devam etmeden doğrulayın

`src/api/istemci.ts` ve üç varlık modülünü yazdınız, `npx tsc -b` hatasız geçiyorsa Ünite 1'in temelini kurmuşsunuz demektir — artık bu isteklerin bekleme/hata durumlarını yönetecek katmanı yazabilirsiniz.

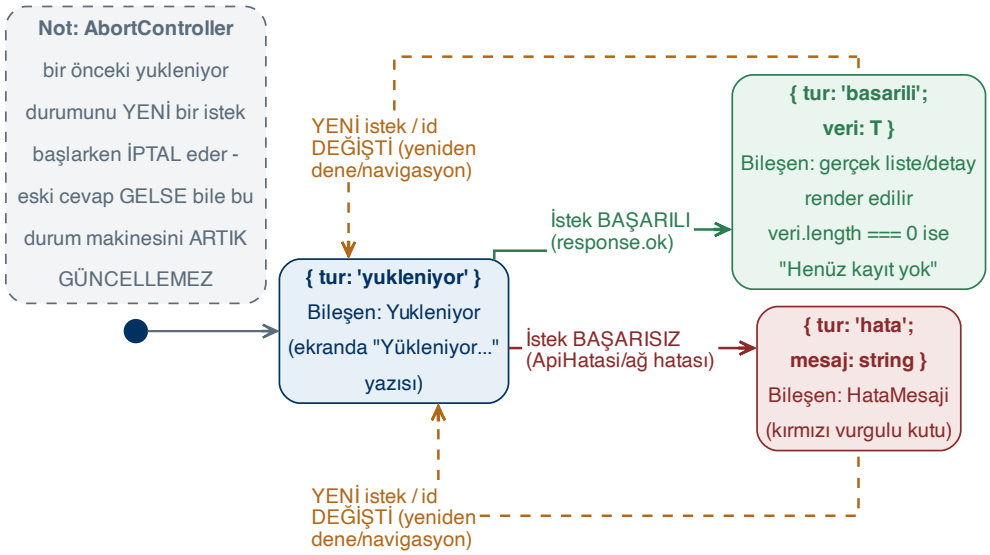
2 Yükleme/Hata/Boş Durumları

(bkz. Ders notu Hafta 7, Ünite 2 — Yükleme/Hata/Boş Durumları.)

Adım 4: `useVeri` hook'unu ve durum bileşenlerini yazın

Ne yapacağız? `src/hooks/useVeri.ts` dosyasını oluşturup `VeriDurumu<T>` ayrımcı birleşim tipini ve `useVeri<T>` özel hook'unu yazacaksınız. Ardından `src/bilesenler/Yukleniyor.tsx` ve `src/bilesenler/HataMesaji.tsx` bileşenlerini yazacaksınız.

Neden? `apiGet` çağrısı anında sonuç vermez — bir süre (ağ gecikmesi) geçer; bu süre boyunca ekranda hiçbir şey göstermezseniz kullanıcı sayfanın donduğunu düşünür. `VeriDurumu<T>`, ortak bir `tur` alanıyla ayırt edilen, her dalın farklı ek alanlar taşıdığı bir tiptir: `durum.tur === 'basarili'` kontrolünden sonra `durum.veri`'ye erişmek güvenlidir, `'yukleniyor'` dalında `durum.veri` hiç yoktur — TypeScript bunu derleme zamanında denetler. `useEffect`'in kendisi `async` olamaz (bir Promise döndüremez); bu yüzden efekt içinde ayrı bir `async` fonksiyon tanımlayıp hemen çağırma kalıbı kullanılır. `AbortController`, bağımlılık değiştiğinde (ör. yeni bir `:id`) ya da bileşen sahneden kalktığında önceki isteği iptal eder — bu sayede bir yarış durumunda (eski cevap yeni cevaptan geç gelirse) state artık eski veriyle güncellenmez.




```ts
// Ortak bir 'tur' alanıyla ayırt edilen, her dalın FARKLI ek alanlar
// taşıdığı bir discriminated union: durum.tur === 'basarili' kontrolünden
// SONRA durum.veri'ye erişmek güvenlidir, 'yukleniyor' dalında durum.veri
// HİÇ YOKTUR — TypeScript bunu derleme zamanında denetler.
export type VeriDurumu<T> =
  | { tur: 'yukleniyor' }
  | { tur: 'hata'; mesaj: string }
  | { tur: 'basarili'; veri: T };
```

```ts
// cagri, AbortController'ın signal'ini ALAN ve bir Promise<T> DÖNDÜREN bir
// fonksiyondur (ör. (signal) => etkinlikleriGetir(undefined, signal)).
// bagimliliklar değiştiğinde (ör. yeni bir :id) efekt YENİDEN çalışır;
// önceki istek controller.abort() ile İPTAL edilir — bu sayede yarış
// durumunda (eski cevap yeni cevaptan GEÇ gelirse) state artık ESKİ
// veriyle GÜNCELLENMEZ.
export function useVeri<T>(
  cagri: (signal: AbortSignal) => Promise<T>,
  bagimliliklar: unknown[],
): VeriDurumu<T> {
```

Adım 4 (devamı)

```
  const [durum, setDurum] = useState<VeriDurumu<T>>({ tur: 'yukleniyor' });

  useEffect(() => {
    const controller = new AbortController();
    setDurum({ tur: 'yukleniyor' });

    // useEffect'in kendisi async OLAMAZ (bir Promise döndüremez); bu
    // yüzden efekt içinde AYRI bir async fonksiyon tanımlanıp HEMEN
    // çağrılır.
    async function veriyiGetir() {
      try {
        const veri = await cagri(controller.signal);
        setDurum({ tur: 'basarili', veri });
      } catch (hata) {
        if (hata instanceof DOMException && hata.name === 'AbortError') {
          // İstek bilinçli olarak iptal edildi (yeni bağımlılık ya da
          // unmount) — state'i GÜNCELLEMEDEN sessizce çık.
          return;
        }
        const mesaj =
          hata instanceof ApiHatasi
            ? typeof hata.detail === 'string'
              ? hata.detail
              : hata.message
            : hata instanceof Error
              ? hata.message
              : 'Bilinmeyen bir hata oluştu.';
        setDurum({ tur: 'hata', mesaj });
      }
    }

    veriyiGetir();

    return () => controller.abort();
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, bagimliliklar);

  return durum;
}
```

```
// VeriDurumu<T>'nin 'yukleniyor' dalı için gösterilen basit gösterge.
export function Yukleniyor() {
  return <p className="sayfa-yukleniyor">Yükleniyor…</p>;
}
```

```
// VeriDurumu<T>'nin 'hata' dalı için gösterilen kırmızı vurgulu kutu.
interface HataMesajiProps {
  mesaj: string;
}

export function HataMesaji({ mesaj }: HataMesajiProps) {
  return <p className="sayfa-hata">{mesaj}</p>;
}
```

Adım 4 (devamı)

```
>_ Terminal
$ npx tsc -b
```

Beklenen sonuç: Komut hatasız biter. Bu üç dosyayı da henüz hiçbir sayfa kullanmıyor — tarayıcıda görünür bir değişiklik yoktur; bunları Bölüm 4'te yazacağınız sayfa bileşenlerinde kullanacaksınız.

Ortak Hata / Tuzak

`useEffect` içinde `controller.abort()` çağıran temizleme fonksiyonunu (dizinin son satırı) unutmak, yarış durumunu hiç önlemez — `AbortController` oluşturmak tek başına yeterli değildir, efektin `return`'ünde mutlaka `abort()` çağırılmalıdır.

3 CORS İlk Temas

(bkz. Ders notu Hafta 7, Ünite 3 — CORS İlk Temas.)

Adım 5: İlk `fetch` denemenizi yapıp CORS engelini gözlemleyin

Ne yapacağız? `src/main.tsx`'in en altına, sadece gözlem amacıyla, Bölüm 1'de yazdığınız `etkinlikleriGetir` ve `etkinlikGetir` fonksiyonlarını çağıran geçici bir test bloğu ekleyeceksiniz. Dosyayı kaydedip tarayıcıda DevTools'un Console sekmesini açacaksınız.

Neden? Bugüne kadar hiç `fetch` denemediniz — bu, uygulamanızın sunucuya yapacağı ilk gerçek ağ isteğidir. Tarayıcı, sayfanızın kökeni (`http://localhost:5173`) ile sunucunun kökeni (`http://127.0.0.1:8000`) farklı olduğu için (yalnızca port farkı bile yeterlidir) varsayılan olarak cevabı JavaScript kodunuza okutturmaz — bu aynı-köken politikasıdır, bir güvenlik önlemidir, bir hata değildir.

```
// GECICI TEST -- CORS gozlemi icin, birazdan silinecek
import { etkinlikleriGetir, etkinlikGetir } from './api/etkinlikler.ts';

etkinlikleriGetir()
  .then((veri) => console.log('Etkinlikler:', veri))
  .catch((hata) => console.error('istek basarisiz (liste):', hata));

etkinlikGetir(999)
  .then((veri) => console.log('999:', veri))
  .catch((hata) => console.error('istek basarisiz (999):', hata));
```

Beklenen sonuç: Konsolda tarayıcının kendi kırmızı "CORS policy" mesajı ve hemen ardından sizin yazdığınız `console.error` çıktısı görünür — iki çağrı da aynı şekilde reddedilir, çünkü `fetch` bu durumda bir ağ hatası (Promise reddi) üretir; `response.ok/status` alanlarına hiç erişilemez.

Adım 5 (devamı)

Beklenen Çıktı

```
Access to fetch at 'http://127.0.0.1:8000/etkinlikler' from origin
'http://localhost:5173' has been blocked by CORS policy: No
'Access-Control-Allow-Origin' header is present on the requested resource.
istek basarisiz (liste): TypeError: Failed to fetch
istek basarisiz (999): TypeError: Failed to fetch
```

Adım 6: `CORSMiddleware`'i ekleyip engeli kaldırın

Ne yapacağız? `sunucu` projenizde `app/main.py` dosyasının `uygulama_olustur` bloğundan hemen sonra, `kok_rota` tanımından önce, `CORSMiddleware` bloğunu ekleyeceksiniz.

Neden? `CORSMiddleware`, sunucunun cevaplarına "bu cevabı `localhost:5173` okuyabilir" başlığını ekler — `allow_origins` bilinçli olarak yalnızca geliştirme adresini içerir; `["*"]` (herkese izin) üretim ortamında bir güvenlik riskidir (ayrıntı Hafta 11). `allow_methods=["*"]` ve `allow_headers=["*"]`, Bölüm 4'te `POST/PATCH` gibi "basit olmayan" isteklerin göndereceği ön uçuş (*preflight*) `OPTIONS` isteğine de sunucunun doğru cevap vermesini sağlar.

```python
# Bu haftanın TEK sunucu değişikliği: tarayıcı, Vite geliştirme
# sunucusunun (http://localhost:5173) origin'inden bu API'nin (8000)
# origin'ine fetch attığında varsayılan olarak cevabı JS koduna
# okutturmaz (aynı-origin politikası). CORSMiddleware, sunucunun
# cevaplarına "bu cevabı localhost:5173 okuyabilir" başlığını (ve
# gerektiğinde preflight OPTIONS cevabını) ekler. allow_origins bilinçli
# olarak yalnızca geliştirme adresini içerir — "*" (herkese izin) üretim
# ortamında güvenlik riski taşır (ayrıntı Hf.11).
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Beklenen sonuç: Terminal A'daki `--reload` bu değişikliği otomatik yakalayıp sunucuyu yeniden başlatır (bellekteki veri — şu an zaten boş — bu sırada sıfırlanır).

Beklenen Çıktı

```
WARNING:  WatchFiles detected changes in 'app/main.py'. Reloading...
INFO:     Application startup complete.
```

Tarayıcı sekmesine dönüp sayfayı yenileyin (aynı geçici test bloğu `main.tsx`'te hâlâ duruyor, sayfa her yenilendiğinde tekrar çalışır) ve konsolu tekrar inceleyin.
Beklenen sonuç: Artık kırmızı bir CORS hatası yoktur. İlk çağrı boş bir dizi döner (depo henüz boş), ikinci çağrı ise `apiGet`'in içindeki `response.ok` kontrolü sayesinde bir `ApiHatasi` (`status: 404`) fırlatır — `fetch`'in kendisi bu isteği artık reddetmez, sunucu bir cevap üretir, yalnızca `response.ok` `false`'dur.

Beklenen Çıktı

```
Etkinlikler: []
istek basarisiz (999): ApiHatasi: API hatası: 404
```

Adım 6 (devamı)

DevTools'un Network sekmesinde `GET /etkinlikler` isteğine tıklayıp `Response Headers` bölümünü açın.

Beklenen sonuç: `access-control-allow-origin: http://localhost:5173` başlığını görürsünüz — tarayıcının cevabı JS kodunuza açmasının nedeni tam olarak bu başlıktır. Gözlemi bitirince `main.tsx`'in sonundaki geçici test bloğunu silin — Bölüm 4'te bu isteği gerçek sayfa bileşenleri yapacak.

Ortak Hata / Tuzak

Tarayıcıyı `http://127.0.0.1:5173` (localhost yerine `127.0.0.1`) ile açmak, `allow_origins` listesindeki `http://localhost:5173` ile tam eşleşmediği için aynı CORS hatasını yeniden tetikler — port aynı olsa bile `localhost` ile `127.0.0.1` farklı köken sayılır; Vite'ı her zaman `localhost:5173` adresinden açtığınızdan emin olun.

Adım 7: `tests/test_cors.py` dosyasını yazın

Ne yapacağız? `sunucu` projenizde `tests/test_cors.py` dosyasını oluşturup `test_cors_izinli_origin` ve `test_cors_izinsiz_origin` testlerini yazacak, ardından tüm test paketini çalıştıracaksınız.

Neden? İlk test, `Origin: http://localhost:5173` başlığıyla gönderilen bir isteğin cevabında `access-control-allow-origin` başlığının var olduğunu doğrular. İkinci test önemli bir ayrımı kanıtlar: izinsiz bir kökenle (`http://localhost:4000`) gönderilen istek sunucuda yine de `200` ile çalışır — CORS bir sunucu yetkilendirmesi değildir, yalnızca tarayıcının cevabı saklamasını sağlayan bir başlık mekanizmasıdır; izinsiz kökende yalnızca bu başlık eksik olur, cevabın kendisi değil.

```python
def test_cors_izinli_origin(client):
    cevap = client.get(
        "/etkinlikler", headers={"Origin": "http://localhost:5173"}
    )
    assert cevap.status_code == 200
    assert cevap.headers["access-control-allow-origin"] == "http://localhost:5173"
```

```python
def test_cors_izinsiz_origin(client):
    cevap = client.get(
        "/etkinlikler", headers={"Origin": "http://localhost:4000"}
    )
    # İstek sunucuda yine de ÇALIŞIR (200 döner) — CORS bir sunucu
    # yetkilendirmesi DEĞİL, tarayıcının cevabı SAKLAMASINI sağlayan bir
    # başlık mekanizmasıdır; izinsiz origin'de yalnızca bu başlık EKSİK
    # olur, tarayıcı cevabı JS koduna ULAŞTIRMAZ.
    assert cevap.status_code == 200
    assert "access-control-allow-origin" not in cevap.headers
```

```
>_ Terminal

$ python -m pytest -q
```

Beklenen sonuç: Hafta 6'nın 9 testi + bu haftanın 2 CORS testi = en az 11 test hatasız geçer (Hafta 6'da `test_koltuklar.py`'yi kendiniz eklediyseniz bu sayı 12'dir).

Adım 7 (devamı)

Beklenen Çıktı

```
...........                                                  [100%]
11 passed, 2 warnings in 0.03s
```

Kontrol Noktası — devam etmeden doğrulayın

CORS hatasını önce gördüyseniz, `CORSMiddleware`'i ekledikten sonra aynı isteğin başarılı olduğunu ve `access-control-allow- origin` başlığını gözlemlediyseniz, `python -m pytest -q` çıktısında en az `11 passed` gördüyseniz, Ünite 3'ün temelini kurmuşsunuz demektir — artık uygulamanızı gerçek, çok sayfalı bir yapıya bölebilirsiniz.

4 React Router: Sayfalar, Parametreli Rotalar

(*bkz. Ders notu Hafta 7, Ünite 4 — React Router: Sayfalar, Parametreli Rotalar.*)

Adım 8: `react-router`'ı kurup ortak sayfa düzenini (`Duzen`) yazın

Ne yapacağız? `istemci-react` klasöründe `react-router` paketini kuracak; `src/bilesenler/Duzen.tsx` dosyasını oluşturacak; `src/style.css`'e yeni sayfa stillerini ekleyecek; son olarak `src/bilesenler/EtkinlikKarti.tsx`'in başlığını bir `Link` ile saracaksınız.

Neden? `Duzen` bileşeni bir nav çubuğu ("Etkinlikler"/ "Biletlerim" bağlantıları) içeren, içinde bir `<Outlet />` bulunduran bir sarmal bileşendir — `Outlet`, o andaki eşleşen alt rotanın render edileceği yer tutucusudur. `Link`, normal bir `<a href="...">` etiketinin aksine tıklandığında sunucuya yeni bir HTTP isteği göndertmez — yalnızca adres çubuğunu güncelleyip eşleşen bileşeni render eder ("gezinme" tarayıcıdan bağımsız, JavaScript içinde gerçekleşir — Hafta 1 Ünite 3'ün SPA tanımının somut kanıtı).

Terminal

```
$ npm install react-router@^8.3.1
```

```tsx
// Tüm sayfaların ORTAK çerçevesi: bir nav çubuğu (her zaman görünür) ve
// altında <Outlet /> — o ANDA eşleşen alt rotanın (EtkinliklerSayfasi,
// EtkinlikDetaySayfasi, BiletlerimSayfasi, BulunamadiSayfasi) render
// EDİLECEĞİ yer tutucu. Link, normal bir <a> etiketinin AKSİNE tıklandığında
// sunucuya YENİ bir HTTP isteği göndertmez — yalnızca adres çubuğunu
// güncelleyip eşleşen bileşeni render eder.
export function Duzen() {
  return (
    <div className="duzen">
      <nav className="duzen-nav">
        <span className="duzen-baslik">KTÜ Etkinlik ve Bilet Sistemi</span>
        <Link to="/">Etkinlikler</Link>
        <Link to="/biletlerim">Biletlerim</Link>
      </nav>
      <main>
        <Outlet />
      </main>
    </div>
```

Adım 8 (devamı)

```
  );
}
```

```
.duzen-nav {
  display: flex;
  align-items: center;
  gap: 1.5rem;
  padding: 1rem 0;
  margin-bottom: 1rem;
  border-bottom: 2px solid var(--renk-lacivert);
}

.duzen-baslik {
  font-weight: bold;
  color: var(--renk-lacivert);
  margin-right: auto;
}

.duzen-nav a {
  color: var(--renk-mavi);
  text-decoration: none;
  font-weight: bold;
}

.duzen-nav a:hover {
  text-decoration: underline;
}

.sayfa-yukleniyor {
  padding: 1rem;
  color: var(--renk-gri);
  font-style: italic;
}

.sayfa-hata {
  padding: 0.75rem 1rem;
  margin: 1rem 0;
  background-color: var(--renk-acikgri);
  border-left: 4px solid var(--renk-bordo);
  color: var(--renk-bordo);
  font-weight: bold;
}

.biletlerim-arama {
  display: flex;
  align-items: flex-end;
  gap: 1rem;
  margin-bottom: 1rem;
}

.biletlerim-arama label {
  display: flex;
  flex-direction: column;
  gap: 0.25rem;
}

.biletlerim-arama input[type='text'] {
```

Adım 8 (devamı)

```
  font: inherit;
  padding: 0.5rem;
  border: 1px solid var(--renk-gri);
  border-radius: 0.25rem;
}

.biletlerim-arama button {
  font: inherit;
  padding: 0.5rem 1rem;
  border: none;
  border-radius: 0.25rem;
  background-color: var(--renk-mavi);
  color: #FFFFFF;
  cursor: pointer;
}

.biletlerim-liste {
  list-style: none;
  padding: 0;
  display: flex;
  flex-direction: column;
  gap: 0.5rem;
}

.biletlerim-liste li {
  padding: 0.75rem 1rem;
  background-color: var(--renk-acikgri);
  border-radius: 0.25rem;
}

.bulunamadi {
  padding: 1.5rem 0;
  text-align: center;
}
```

```
// Hf.2'nin article.etkinlik-karti HTML yapısının React karşılığı;
// biletFiyati 0 ise üçlü operatörle "Ücretsiz" rozeti gösterilir. Başlık
// artık bir Link'tir (Hf.7): tıklandığında sunucuya YENİ bir HTTP isteği
// GİTMEZ, yalnızca adres çubuğu /etkinlikler/{id}'ye GÜNCELLENİR ve
// EtkinlikDetaySayfasi render edilir.
export function EtkinlikKarti({ etkinlik }: EtkinlikKartiProps) {
  return (
    <article className="etkinlik-karti">
      <h3>
        <Link to={`/etkinlikler/${etkinlik.id}`}>{etkinlik.baslik}</Link>
      </h3>
      <p>
        {etkinlik.tarih} · {etkinlik.mekan}
      </p>
      <p>
        Kapasite: {etkinlik.kapasite} ·{' '}
        {etkinlik.biletFiyati === 0 ? (
          <span className="rozet-ucretsiz">Ücretsiz</span>
        ) : (
          `Bilet: ${etkinlik.biletFiyati} TL`
        )}
      </p>
```

Adım 8 (devamı)

```
    </article>
  );
}
```

Beklenen sonuç: `npm install` `package.json`'a `"react-router": "^8.3.1"` ekler, hatasız biter. `npx tsc -b` de hatasız kalır — tip denetimi router bağlamını (bir `<BrowserRouter>` içinde olup olmadığınızı) kontrol etmez.

> **Dikkat**
>
> `EtkinlikKarti.tsx`'i kaydettiğiniz an, tarayıcıdaki açık sayfa (hâlâ eski `App.tsx` ile çalışıyor) kırmızı bir çalışma zamanı hatasıyla karşılaşabilir ("`useHref()` yalnızca bir `<Router>` bağlamında kullanılabilir" benzeri bir mesaj) — bu beklenen bir ara durumdur: `Link`, bir router bağlamı gerektirir, ama bu bağlamı Bölüm 4'ün son adımında `App.tsx`'i yeniden yazınca kuracaksınız. Paniklemeyin, bir sonraki adımlarda yeni dosyalar yazmaya devam edin — kırmızı ekran o ana kadar normaldir.

Adım 9: `EtkinliklerSayfasi` ve `BulunamadiSayfasi` bileşenlerini yazın

Ne yapacağız? `src/sayfalar` klasörünü oluşturup içine `EtkinliklerSayfasi.tsx` ve `BulunamadiSayfasi.tsx` dosyalarını ekleyeceksiniz.

Neden? `EtkinliklerSayfasi`, `useVeri<Etkinlik[]>` ile Hafta 6'nın `GET /etkinlikler`'ini çeker; `durum.tur`'a göre `Yukleniyor`/`HataMesaji`/gerçek liste arasında seçim yapar (boş liste durumu `EtkinlikListesi`'nin kendi içinde ele alınır). `BulunamadiSayfasi` ise `Routes` içindeki hiçbir yolla eşleşmeyen her URL için (`path="*"` ile, en sonda tanımlanan bir "joker" yol) render edilir — adres yazım olarak geçerlidir, ama orada tanımlı bir rota yoktur.

```
// '/' rotası — useVeri<Etkinlik[]> ile Hf.6'nın GET /etkinlikler'ini
// çeker; üç durumdan hangisindeyse (yukleniyor/hata/basarili) ona göre
// render eder. Boş liste durumu EtkinlikListesi'nin KENDİ İÇİNDE (Hf.4'ten
// aynen) ele alınır ("Henüz etkinlik yok.").
export function EtkinliklerSayfasi() {
  const durum = useVeri<Etkinlik[]>(
    (signal) => etkinlikleriGetir(undefined, signal),
    [],
  );

  if (durum.tur === 'yukleniyor') {
    return <Yukleniyor />;
  }
  if (durum.tur === 'hata') {
    return <HataMesaji mesaj={durum.mesaj} />;
  }

  return (
    <section className="bolum">
      <h2>Etkinlikler</h2>
      <EtkinlikListesi etkinlikler={durum.veri} />
    </section>
  );
}
```

```
// Routes içindeki HİÇBİR yolla eşleşmeyen (path="*") her URL için render
```

Adım 9 (devamı)

```
// edilir — adres YAZIM olarak geçerlidir, ama orada tanımlı bir rota YOKTUR.
export function BulunamadiSayfasi() {
  return (
    <section className="bulunamadi">
      <h2>404 — Sayfa bulunamadı</h2>
      <p>Aradığınız adres için tanımlı bir sayfa yok.</p>
      <Link to="/">Anasayfaya dön</Link>
    </section>
  );
}
```

Beklenen sonuç: `npx tsc -b` Hatasız kalır. Bu iki dosyayı da henüz hiçbir `Route` kullanmıyor — tarayıcıdaki kırmızı hata hâlâ açıktır, bu normaldir.

Adım 10: `EtkinlikDetaySayfasi`'nı yazıp `BiletFormu`'yu gerçek API'ye bağlayın

Ne yapacağız? `src/sayfalar/EtkinlikDetaySayfasi.tsx` dosyasını oluşturacak; `useParams` ile URL'den `id`'yi okuyup `useVeri` ile etkinlik+koltuk verisini birlikte çekecek, `SepetProvider` içinde koltuk planı/sepet özeti/geri sayım/bilet formunu birleştirecek ve koltuk tıklamasında iyimser güncellemeyi uygulayacaksınız. Ardından `src/bilesenler/BiletFormu.tsx`'teki gönderim fonksiyonunu (`submitEt`) artık `biletOlustur` API çağrısı yapacak, `409` cevabında hata gösterecek, başarıda `useNavigate` ile `/biletlerim`'e yönlendirecek biçimde güncelleyeceksiniz.

Neden? `useParams`'tan gelen `id` her zaman bir `string`'dir (URL bir metindir) — `apiGet`'e geçirmeden önce `Number(id)` ile sayıya çevrilir. `bagimliliklar` dizisindeki `etkinlikId` sayesinde `/etkinlikler/3`'ten `/etkinlikler/7`'ye hızlıca geçildiğinde efekt yeniden çalışır, önceki istek iptal edilir. İyimser güncelleme deseni: bir koltuğa tıklandığında `Koltuk` bileşeni (Hafta 5'ten aynen) onu anında "seçili" gösterir — sunucu cevabı beklenmez; ardından bir `useEffect` sepete yeni eklenen koltuğu izleyip bir `PATCH /koltuklar/{id}` isteği gönderir: istek başarılıysa hiçbir şey değişmez (iyimser durum zaten doğruydu), koltuk bu arada başkası tarafından alınmışsa (`409`) koltuk sepetten geri çıkarılır ve bir hata mesajı gösterilir — "önce iyimser göster, hata gelirse geri al" deseni. `BiletFormu`'daki `e.preventDefault()` ve doğrulama mantığı aynen kalır; tek fark, artık her seçili koltuk için gerçek bir `POST /biletler` isteğinin gönderilmesi ve sunucunun `409` dönmesi durumunda bunun kullanıcıya gösterilmesidir (Hafta 6'da yalnızca `curl` ile gözlenen bu iş kuralı, bugün gerçek arayüze yansır).

```
// '/etkinlikler/:id' rotası — useParams'tan gelen id HER ZAMAN bir string'dir
// (URL bir metindir), apiGet'e geçirilmeden önce Number(id) ile sayıya
// çevrilir. useVeri, etkinlik VE koltuk verisini TEK bir çağrıda
// (Promise.all) BİRLİKTE çeker; bagimliliklar dizisindeki etkinlikId
// SAYESİNDE /etkinlikler/3'ten /etkinlikler/7'ye HIZLICA geçildiğinde efekt
// YENİDEN çalışır, önceki istek AbortController ile İPTAL edilir (yarış
// durumu önlenir).
export function EtkinlikDetaySayfasi() {
  const { id } = useParams<{ id: string }>();
  const etkinlikId = Number(id);

  const durum = useVeri<EtkinlikDetayVerisi>(
    async (signal) => {
      const [etkinlik, koltuklar] = await Promise.all([
        etkinlikGetir(etkinlikId, signal),
        koltuklariGetir(etkinlikId, undefined, signal),
```

Adım 10 (devamı)

```
    ]);
    return { etkinlik, koltuklar };
  },
  [etkinlikId],
 );

 if (durum.tur === 'yukleniyor') {
  return <Yukleniyor />;
 }
 if (durum.tur === 'hata') {
  return <HataMesaji mesaj={durum.mesaj} />;
 }

 return (
  <SepetProvider biletFiyati={durum.veri.etkinlik.biletFiyati}>
   <EtkinlikDetayIcerik
    etkinlik={durum.veri.etkinlik}
    ilkKoltuklar={durum.veri.koltuklar}
   />
  </SepetProvider>
 );
}
```

```
// İyimser güncelleme: Koltuk bileşeni (Hf.5'ten aynen) bir koltuğa
// tıklandığında koltukSec() ile onu SEPETE hemen ekler — kullanıcı bunu
// ANINDA "seçili" olarak görür, sunucu cevabı BEKLENMEZ. Bu efekt sepete
// YENİ eklenen koltuğu izleyip sunucuya bir PATCH /koltuklar/{id} isteği
// (koltukGuncelle) gönderir: istek başarılıysa iyimser durum KORUNUR;
// koltuk bu ARADA başka bir kullanıcı tarafından alınmışsa (409) koltuk
// sepetten GERİ ÇIKARILIR (koltukSec tekrar çağrılarak) ve yerel koltuk
// listesi "dolu" olarak GÜNCELLENİR.
useEffect(() => {
 const guncelIdler = new Set(secilenKoltuklar.map((k) => k.id));
 const yeniSecilen = secilenKoltuklar.find(
  (k) => !izlenenIdler.current.has(k.id),
 );
 izlenenIdler.current = guncelIdler;

 if (!yeniSecilen) return;

 koltukGuncelle(yeniSecilen.id, 'secili')
  .then(() => setKoltukHatasi(null))
  .catch((hata) => {
   koltukSec(yeniSecilen); // tekrar çağırmak sepetten ÇIKARIR (toggle)
   if (hata instanceof ApiHatasi && hata.status === 409) {
    setKoltukHatasi('Bu koltuk bu arada başka bir kullanıcı tarafından alındı.');
    setKoltuklar((onceki) =>
     onceki.map((k) =>
      k.id === yeniSecilen.id ? { ...k, durum: 'dolu' } : k,
     ),
    );
   } else {
    setKoltukHatasi('Koltuk güncellenirken bir hata oluştu.');
   }
  });
 // eslint-disable-next-line react-hooks/exhaustive-deps
}, [secilenKoltuklar]);
```

Adım 10 (devamı)

```
return (
  <>
    <section className="bolum">
      <h2>{etkinlik.baslik}</h2>
      <p>
        {etkinlik.tarih} · {etkinlik.mekan}
      </p>
    </section>
    <section className="bolum">
      <h2>Koltuk Planı</h2>
      {koltukHatasi && <HataMesaji mesaj={koltukHatasi} />}
      <KoltukPlani koltuklar={koltuklar} />
    </section>
    <section className="bolum">
      <h2>Sepet</h2>
      <SepetOzeti />
      <GeriSayim />
      <BiletFormu
        onBiletlerOlustu={(yeniBiletler) =>
          setBiletler((onceki) => [...onceki, ...yeniBiletler])
        }
      />
      {biletler.length > 0 && (
        <p className="bilet-ozet">Oluşturulan bilet sayısı: {biletler.length}</p>
      )}
    </section>
  </>
);
```

```
// İlk satır e.preventDefault(): aksi halde tarayıcı VARSAYILAN
// davranışıyla sayfayı yeniden yükler ve tüm React state'i sıfırlanır.
// Doğrulama AYNI kalır (ad boş mu, sepette hiç koltuk yok mu); ama
// artık her seçili koltuk için GERÇEK bir POST /biletler isteği
// (biletOlustur) gönderilir. Sunucu bu ARADA koltuğu BAŞKASINA
// satmışsa 409 döner — apiPost bunu bir ApiHatasi olarak fırlatır,
// burada YAKALANIP kullanıcıya GÖSTERİLİR (Hf.6'da yalnızca curl ile
// gözlenen bu iş kuralı artık GERÇEK arayüze yansır). Başarıda
// useNavigate ile /biletlerim'e yönlendirilir.
async function submitEt(e: FormEvent<HTMLFormElement>) {
  e.preventDefault();

  if (form.ad.trim() === '') {
    setHata('Ad alanı boş olamaz.');
    return;
  }
  if (secilenKoltuklar.length === 0) {
    setHata('Sepette en az bir koltuk seçili olmalı.');
    return;
  }

  try {
    const yeniBiletler = await Promise.all(
      secilenKoltuklar.map((koltuk) =>
        biletOlustur({ koltukId: koltuk.id, kullaniciAdi: form.ad }),
      ),
    );

```

Adım 10 (devamı)

```
    onBiletlerOlustu(yeniBiletler);
    sepetiTemizle();
    setHata(null);
    const kullaniciAdi = form.ad;
    setForm(BOS_FORM);
    navigate(`/biletlerim?kullaniciAdi=${encodeURIComponent(kullaniciAdi)}`);
  } catch (e) {
    if (e instanceof ApiHatasi && e.status === 409) {
      setHata('Seçtiğiniz koltuklardan biri bu arada başkasına satıldı.');
    } else {
      setHata('Bilet oluşturulurken bir hata oluştu. Lütfen tekrar deneyin.');
    }
  }
}
```

Beklenen sonuç: `npx tsc -b` Hatasız kalır — `BiletFormu` ve `SepetContext/useSepet` ile ilgili geri kalan kod (`bilet_form_tipi`, `bilet_form_state`, `bilet_form_render`) Hafta 5'ten aynen kalır, sadece `submitEt` fonksiyonunun içi değişir.

Ortak Hata / Tuzak

Koltuk tıklamasında önce `koltukGuncelle` isteğinin cevabını `await` edip ancak ondan sonra arayüzü güncellemek, iyimser güncellemenin tüm amacını ortadan kaldırır — kullanıcı her tıklamada ağ gecikmesi kadar bekler. Koltuğu önce `useSepet`'in `koltukSec`'i ile anında seçip, sunucu isteğini ayrı bir `useEffect`'te, arka planda göndermeniz gerekir.

Adım 11: `BiletlerimSayfasi`'nı yazıp `App.tsx`'i router ile yeniden yazın

Ne yapacağız? `src/sayfalar/BiletlerimSayfasi.tsx` dosyasını oluşturup URL sorgu parametresinden (`kullaniciAdi`) okunan kullanıcının biletlerini listeleyeceksiniz. Ardından `src/ App.tsx`'i tamamen yeniden yazıp `BrowserRouter+Routes` ile dört sayfayı (`Duzen` altında `index`, `"etkinlikler/:id"`, `"biletlerim"`, `path="*"`) bağlayacak; son olarak `src/veri.ts` dosyasını sileceksiniz.

Neden? `useSearchParams`, URL'nin sorgu kısmını (`? kullaniciAdi=...`) okuyup yazmanızı sağlar — Hafta 10'a kadar gerçek bir oturum yoktur, bu yüzden kullanıcı adı basit bir arama formuyla da değiştirilebilir. `App.tsx`'teki `Routes` içindeki `Route`'lar sırayla değerlendirilir, ilk eşleşen yol render edilir — bu yüzden `path="*"` en sonda durur. `Route element=<Duzen />` İçine yerleştirilen iç `Route`'lar hep aynı nav çubuğunun altında görünür. Bu adımdan sonra tarayıcıdaki kırmızı hata kaybolacaktır — `Link/ useNavigate` artık bir `<BrowserRouter>` bağlamı içindedir.

```
// '/biletlerim' rotası — kullaniciAdi URL sorgu parametresinden okunur
// (ör. BiletFormu'nun yönlendirdiği /biletlerim?kullaniciAdi=Ayşe%20Yılmaz);
// Hf.10'a kadar gerçek bir oturum YOKTUR, bu yüzden kullanıcı adı bir form
// ile de DEĞİŞTİRİLEBİLİR. kullaniciAdi boşken hiç istek ATILMAZ.
export function BiletlerimSayfasi() {
  const [searchParams, setSearchParams] = useSearchParams();
  const kullaniciAdi = searchParams.get('kullaniciAdi') ?? '';
  const [taslakAd, setTaslakAd] = useState(kullaniciAdi);

  function ara(e: FormEvent<HTMLFormElement>) {
    e.preventDefault();
```

Adım 11 (devamı)

```
    setSearchParams(taslakAd.trim() ? { kullaniciAdi: taslakAd.trim() } : {});
  }

  const durum = useVeri<Bilet[]>(
    (signal) =>
      kullaniciAdi ? biletleriGetir(kullaniciAdi, signal) : Promise.resolve([]),
    [kullaniciAdi],
  );

  return (
    <section className="bolum">
      <h2>Biletlerim</h2>
      <form className="biletlerim-arama" onSubmit={ara}>
        <label>
          Kullanıcı adı
          <input
            type="text"
            value={taslakAd}
            onChange={(e) => setTaslakAd(e.target.value)}
          />
        </label>
        <button type="submit">Biletleri Göster</button>
      </form>

      {kullaniciAdi === '' && <p>Biletlerinizi görmek için kullanıcı adınızı girin.</p>}
      {kullaniciAdi !== '' && durum.tur === 'yukleniyor' && <Yukleniyor />}
      {kullaniciAdi !== '' && durum.tur === 'hata' && <HataMesaji mesaj={durum.mesaj} />}
      {kullaniciAdi !== '' && durum.tur === 'basarili' && (
        durum.veri.length === 0 ? (
          <p>Bu kullanıcı adına ait bilet bulunamadı.</p>
        ) : (
          <ul className="biletlerim-liste">
            {durum.veri.map((bilet) => (
              <li key={bilet.id}>
                Bilet #{bilet.id} — koltuk {bilet.koltukId} — {bilet.durum} (
                {bilet.satinAlmaZamani})
              </li>
            ))}
          </ul>
        )
      )}
    </section>
  );
}
```

```
// Uygulamanın TAMAMI artık BrowserRouter içinde yaşar; Routes içindeki
// Route'lar SIRAYLA değerlendirilir, İLK eşleşen yol RENDER edilir.
// Route element={<Duzen />} İÇİNE yerleştirilen İÇ Route'lar (index,
// "etkinlikler/:id", "biletlerim") HEP AYNI Duzen nav çubuğunun ALTINDA
// görünür (Duzen'in <Outlet /> yer tutucusu SAYESİNDE); path="*" EN SONDA
// tanımlanır, tanımsız her adres için BulunamadiSayfasi render edilir.
export function App() {
  return (
    <BrowserRouter>
      <Routes>
```

Adım 11 (devamı)

```
      <Route element={<Duzen />}>
        <Route index element={<EtkinliklerSayfasi />} />
        <Route path="etkinlikler/:id" element={<EtkinlikDetaySayfasi />} />
        <Route path="biletlerim" element={<BiletlerimSayfasi />} />
        <Route path="*" element={<BulunamadiSayfasi />} />
      </Route>
    </Routes>
  </BrowserRouter>
 );
}
```

Terminal

```
$ rm src/veri.ts
$ npx tsc -b
$ npm run build
```

Beklenen sonuç: Her iki komut da hatasız biter — `veri.ts`'i kullanan son dosya (`App.tsx`) artık onu import etmiyordu, bu yüzden silme işlemi tip hatası üretmez. Tarayıcıdaki sayfa artık kırmızı hata göstermeden, "Etkinlikler" başlıklı boş bir liste ("Henüz etkinlik yok.") ile açılır — veri artık `veri.ts`'in sabit dizisinden değil, sunucudan gelir, sunucu henüz boştur.

Beklenen Çıktı

```
vite v8.2.2 building client environment for production...
transforming...
✓ 97 modules transformed.
rendering chunks...
computing gzip size...
dist/index.html                   0.47 kB │ gzip:  0.31 kB
dist/assets/index-tLIIfOxg.css    3.66 kB │ gzip:  1.10 kB
dist/assets/index-Bvym4Ur4.js   238.25 kB │ gzip: 76.44 kB
✓ built in 73ms
```

Adım 12: Uçtan uca deneyin: sunucuya veri ekleyip tam akışı ve `409` çakışmasını gözlemleyin

Ne yapacağız? `curl` ile sunucuya iki etkinlik ve bir koltuk ekleyecek; tarayıcıda etkinlik listesinden bir karta tıklayıp koltuk seçip bilet alacak; ardından iki tarayıcı sekmesi açıp aynı koltuğu ikisinden de almaya çalışarak `409` çakışmasını ekranda göreceksiniz.

Neden? Bu adım, bugün yazdığınız dört ünitenin (tiplenmiş istemci, üç durumlu model, CORS, router) hepsini tek bir kullanıcı akışında birleştirir — Etkinlik/Bilet Sisteminin ön yüzü nihayet Hafta 6'nın gerçek sunucusuna bağlanmıştır.

Adım 12 (devamı)

Terminal

```
$ curl -s -X POST http://127.0.0.1:8000/etkinlikler \
  -H "Content-Type: application/json" \
  -d '{"baslik": "Bahar Konseri", "tarih": "2026-11-14",
       "mekan": "Açık Hava Amfisi", "kapasite": 500, "biletFiyati": 150}'
$ curl -s -X POST http://127.0.0.1:8000/etkinlikler \
  -H "Content-Type: application/json" \
  -d '{"baslik": "Kariyer Günleri", "tarih": "2026-11-20",
       "mekan": "Kongre Merkezi", "kapasite": 800, "biletFiyati": 50}'
$ curl -s -X POST http://127.0.0.1:8000/etkinlikler/1/koltuklar \
  -H "Content-Type: application/json" \
  -d '{"sira": 1, "koltukNo": 5}'
```

Beklenen sonuç: Üç istek de `id`'si atanmış birer JSON nesnesi döner (`201 Created` örtük — `-s` sessiz modda yalnızca gövdeyi gösterir).

Beklenen Çıktı

```
{"baslik":"Bahar Konseri","tarih":"2026-11-14","mekan":"Açık Hava
Amfisi","kapasite":500,"biletFiyati":150.0,"id":1}
{"baslik":"Kariyer Günleri","tarih":"2026-11-20","mekan":"Kongre
Merkezi","kapasite":800,"biletFiyati":50.0,"id":2}
{"sira":1,"koltukNo":5,"id":1,"etkinlikId":1,"durum":"bos"}
```

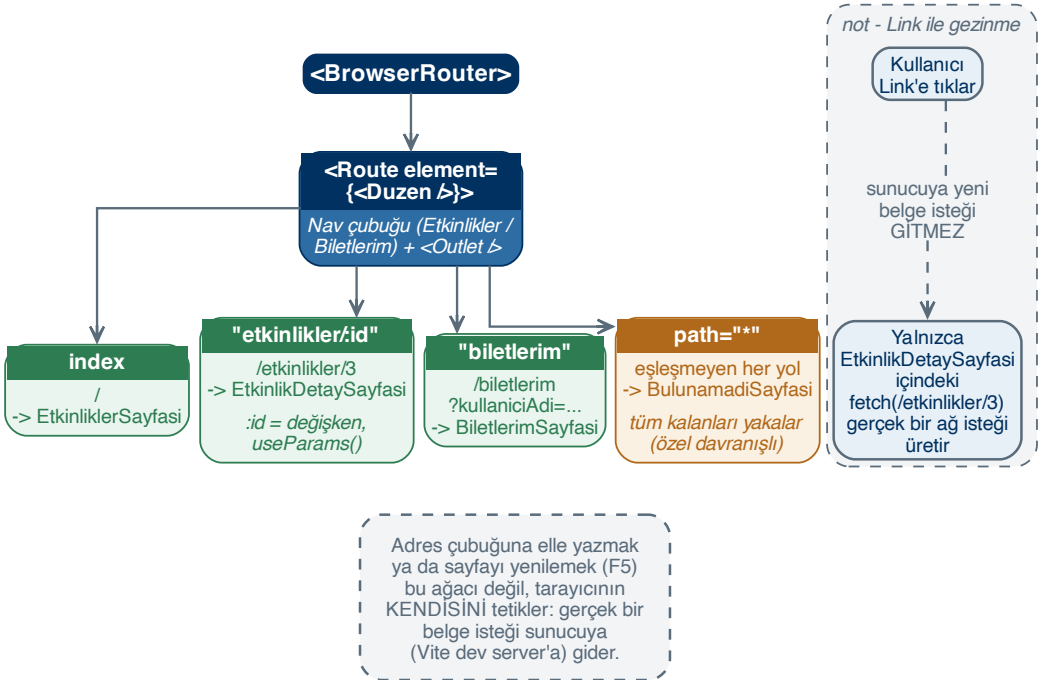
Tarayıcıda `http://localhost:5173`'i yenileyin, "Bahar Konseri" kartına tıklayın, koltuğa tıklayıp seçin, bilet formunu (ad-soyad, kampanya onayı) doldurup gönderin.
Beklenen sonuç: Adres çubuğu sayfa yenilenmeden `/etkinlikler/1`'e değişir; koltuk tıklandığı an "seçili" renge döner; form gönderildiğinde adres çubuğu `/biletlerim?kullaniciAdi=...`'e yönlenir ve yeni oluşturduğunuz bilet listede görünür.

ikinci bir tarayıcı sekmesi açıp aynı `/etkinlikler/1` adresine gidin, aynı (artık `'dolu'` olan) koltuğa tıklayıp formu göndermeyi deneyin.
Beklenen sonuç: Koltuk önce iyimser olarak "seçili" görünür, ama sunucudan `409` cevabı gelince koltuk "dolu" durumuna geri döner ve ekranda "Bu koltuk bu arada başka bir kullanıcı tarafından alındı." benzeri bir hata mesajı belirir. Son olarak adres çubuğuna `http://localhost:5173/xyz` yazıp Enter'a basın — `BulunamadiSayfasi` (404) görünür.

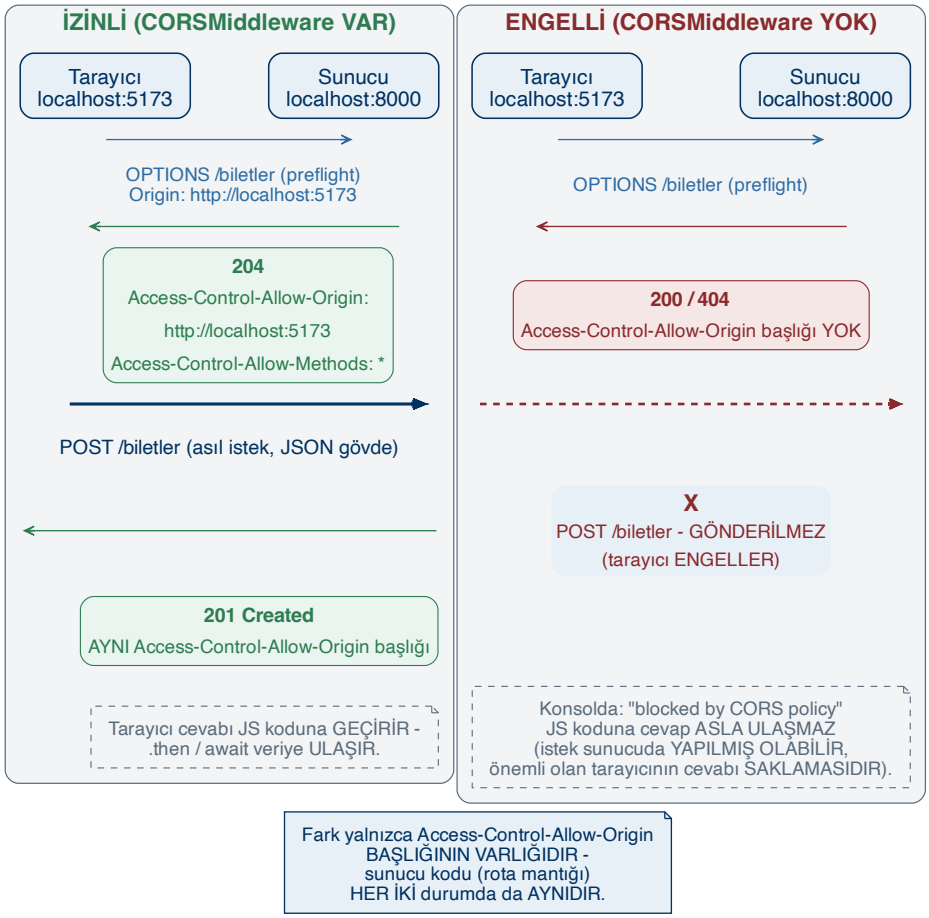
Beklenen Çıktı

```
HTTP/1.1 409 Conflict
{"detail":"Bu koltuk zaten satılmış"}
```

Adım 12 (devamı)



Adres çubuğuna elle yazmak ya da sayfayı yenilemek (F5) bu ağacı değil, tarayıcının KENDİSİNİ tetikler: gerçek bir belge isteği sunucuya (Vite dev server'a) gider.



Fark yalnızca Access-Control-Allow-Origin BAŞLIĞININ VARLIĞIDIR - sunucu kodu (rota mantığı) HER İKİ durumda da AYNIDIR.

Kontrol Noktası — devam etmeden doğrulayın

Etkinlik listesinden bir karta tıklayıp sayfa yenilenmeden detay sayfasına geçtiyseniz, bir koltuk seçip bilet aldıysanız, `/biletlerim`'de bileti gördüyseniz, ikinci sekmede `409` çakışmasını ve tanımsız bir adreste 404 sayfasını gözlemlediyseniz, `npx tsc -b && npm run build` hatasız geçtiyse, bu haftanın dört ünitesini de uygulamışsınız demektir — Etkinlik/Bilet Sisteminin ön yüzü artık gerçek sunucuya bağlıdır.

Adım 13: Kendiniz ekleyin: Etkinlikler sayfasına mekan filtresi ekleyin

Ne yapacağız? `src/sayfalar/EtkinliklerSayfasi.tsx`'e bir `mekan` metin girdisi (`input`) ekleyecek; bu girdinin değerini bir `useState` ile tutup `useVeri`'nin ikinci argümanına (`bagimliliklar` dizisine) ekleyecek ve `etkinlikleriGetir` çağrısına `mekan || undefined` biçiminde geçireceksiniz.

Neden? `etkinlikleriGetir`'in ikinci parametresi zaten Hafta 6'nın `mekan` sorgu parametresine (Bölüm 1'de yazdığınız `etkinlikler_api`'ye bakın) bağlanmıştı — bu görev, sunucu tarafında zaten var olan bu filtreyi arayüze bağlamanızı ister. `mekan`'ı `useVeri`'nin bağımlılık dizisine eklemek, Ünite 2'nin öğrettiği deseni tekrarlar: bağımlılık değiştiğinde efekt yeniden çalışır, önceki istek `AbortController` ile iptal edilir.

Beklenen sonuç: Girdiye tam olarak "Kongre Merkezi" yazdığınızda liste yalnızca "Kariyer Günleri"ni gösterir (sunucu tarafındaki eşleşme tam metin eşleşmesidir, alt dize değil); girdiyi boşalttığınızda her iki etkinlik de tekrar görünür. DevTools Network sekmesinde her değişiklikte yeni bir `GET /etkinlikler?mekan=...` isteği görünür.

İpucu

`BiletlerimSayfasi.tsx`'teki arama formunun (`taslakAd`+ `onSubmit`) yapısı size bir şablon sağlar — isterseniz aynı deseni (bir taslak state + "Filtrele" düğmesi) kullanabilir, isterseniz girdinin `onChange`'ini doğrudan `mekan` state'ine bağlayıp her tuş vuruşunda yeniden çekebilirsiniz.

Sorun Giderme

Belirti	Olası neden	Çözüm
Tarayıcı konsolunda "has been blocked by CORS policy" hatası `CORSMiddleware` eklendikten sonra da devam ediyor	Sayfa `http://127.0.0.1:5173` adresinden açılmış (`localhost` değil) — `allow_origins` listesindeki adresle tam eşleşmiyor	Vite'ı daima `http://localhost:5173` adresinden açtığınızı doğrulayın; gerekirse sekmeyi tamamen kapatıp yeniden açın
`npx tsc -b` "Cannot find module 'react-router'" hatası veriyor	`npm install react-router@^8.3.1` komutu hiç çalıştırılmamış ya da `istemci-react` dışında bir klasörden çalıştırılmış	`istemci-react` klasöründe `package.json`'da `"react-router"` bağımlılığının olduğunu kontrol edip `npm install`'i tekrar çalıştırın
Konsolda yakalanmamış (*uncaught*) bir `AbortError` görünüyor	`useVeri` içindeki `catch` bloğunda `hata instanceof DOMException && hata.name === 'AbortError'` kontrolü eksik	`useVeri.ts`'teki `catch` bloğunun ilk satırının bu kontrolü yapıp erken `return` ettiğini doğrulayın
Etkinlik detay sayfası sürekli "Yükleniyor..." gösterip hiç bitmiyor	`useParams`'tan gelen `id` `Number()` ile çevrilmeden `apiGet`'e geçirilmiş, sunucu `422` dönüyor ve hata yakalanmıyor	`EtkinlikDetaySayfasi`'nda `const etkinlikId = Number(id);` satırının var olduğunu ve `useVeri` çağrısının `etkinlikId`'yi (string `id` değil) kullandığını kontrol edin
Tanımsız bir adrese gidince (ör. `/xyz`) 404 sayfası yerine beyaz boş bir sayfa görünüyor	`path="*"` rotası `Routes` içinde en sonda değil, ya da `Duzen`'i saran `Route`'un dışında tanımlanmış	`App.tsx`'teki `Route`'ların sırasını ve `path="*"`'in `Route element=<Duzen />` içinde, en sonda olduğunu doğrulayın
Bilet formu her koltukta `409` ile reddediliyor, ilk seçimde bile	`sunucu` terminaliniz Hafta 6'dan beri bir kez daha `--reload` ile yeniden başlamış (veri sıfırlanmış) ama tarayıcıdaki koltuk listesi eski (dolu) durumu HÂLÂ gösteriyor	Tarayıcı sekmesini tamamen yenileyip (`F5`) koltuk planının sunucudan taze veriyle yeniden çekildiğini doğrulayın

5 · İzleme — Aktarım — Başka Nerede?

Bugün Etkinlik/Bilet Sisteminin ön yüzünü Hafta 6'nın gerçek sunucusuna bağladınız: `fetch`'in ağ hatası (Promise reddi) ile HTTP hata cevabı (Promise'in normal çözülmesi) arasındaki ayrımı tek bir tiplenmiş istemcide topladınız, bekleme/hata/veri üç durumunu `useVeri<T>` hook'unda birleştirdiniz, tarayıcının aynı-köken politikasını tek bir `CORSMiddleware` bloğuyla aştınız ve uygulamanızı `react-router` ile dört gerçek sayfaya böldünüz. Bugün kurduğunuz `apiGet`/`apiPost`/`ApiHatasi` deseni Hafta 10'un giriş isteğinde, Hafta 13'ün WebSocket öncesi ilk bağlantı isteğinde aynen yeniden kullanılacak; `yukleniyor | hata | veri` üç durumlu modeli Hafta 10'un giriş formunda ("giriş yapılıyor..."/"şifre yanlış"/"başarılı") aynı iskeletle tekrar karşınıza çıkacak; bugün öğrendiğiniz preflight/başlık mekanizması Hafta 11'de bir güvenlik konusu olarak (`allow_origins=["*"]` riskleri ile) daha derin ele alınacak; `Duzen`+`Outlet`+ sayfa iskeletiniz Hafta 10'da `/giris`/`/kayit`/korumalı `/panel` rotalarıyla genişleyecek. Gelecek hafta ise sunucunuzun HÂLÂ geçici olan belleği SQLAlchemy ile gerçek bir veritabanına (SQLite) taşınıp kalıcı hâle gelecek — bugün yazdığınız

`istemci-react` klasörünün tek bir satırı değişmeyecek, çünkü API sözleşmesi (Hafta 6'nın camelCase kararı) aynı kalacaktır.